

Agent Control 3.0.1 Operator Guide

Infrastructure-neutral installation, operation, monitoring and recovery

Release 3.0.1

24 August 2026

Execution may be delegated. Authority is not.

Purpose

Agent Control coordinates durable AI-agent work while retaining human and policy authority. This guide covers installation, configuration, operation, monitoring, recovery, optional Orca execution, Android resources, evidence and safe shutdown.

What Agent Control does

- runs multiple AUTO/MANUAL lanes with priorities;
- persists contracts, batons, leases, events and checkpoints;
- supports handoff, clone and shared-task semantics;
- schedules by resource/provider capability, health, load, locality, cost and confidence;
- manages background/batch work, retries, approvals and human review;
- fences PTY ownership and unconditional human takeover;
- compares independent attempts and records consensus/provenance;
- attaches provider-neutral shared context without replacing Git/test evidence;
- delegates execution through a replaceable contract, including optional Orca;
- monitors configured services, providers, remote resources and Android nodes.

Authority boundary

Agent Control decides placement, priority, ownership, continuation, cancellation, winner selection and recovery validity. An execution substrate may create a process, PTY or worktree, but it cannot grant itself a lease or bypass a takeover fence.

Requirements

Use Node.js 20+, npm, Git and Bash. Install Orca, SSH, Android/Termux or model services only when those optional facilities are needed. The source release has no required machine name, provider endpoint, port, GPU or network overlay.

Install

```
git clone <repository-url>
cd agent-control
npm install
mkdir -p .agent-control
cp config/agent-control.example.json .agent-control/config.json
npm run check
```

Windows PowerShell equivalent:

```
git clone <repository-url>
Set-Location agent-control
npm install
New-Item -ItemType Directory -Force .agent-control
Copy-Item config/agent-control.example.json .agent-control/config.json
npm run check
```

Do not copy credentials into configuration. Use the named credential environment variable or an operating-system secret/service manager.

Configure

The default configuration path is `.agent-control/config.json`. Override it with `AGENT_CONTROL_CONFIG`; override runtime state with `AGENT_CONTROL_STATE_DIR`.

Resources combine a stable ID, platform, transport and capabilities. Providers describe the API/CLI capability and optional qualification model. Services describe an explicit health URL and optional start recipe. Lanes describe ID, name, directory, priority and mode.

The example file illustrates local, SSH and Android resources, an alternate provider port, an optional managed service and two lanes. Replace every example endpoint. Empty collections are valid.

See [Integrations and credentials](integrations-and-credentials.md) before configuring an LLM, Codex, ChatKit, Orca or Android resource. In particular, `chatgpt-window` is not included in v3.0.1; a Responses provider URL must be served by an external process.

The first configuration file is only an explicit safety boundary. It is not a discovery database: providers, resources, services and lanes become meaningful only when the operator adds real endpoints, transports, capabilities and commands. The current TUI probes configured integrations and maintains lane state, but does not submit ordinary lane prompts to an LLM.

Use [Concepts and terms](concepts.md) for the plain-language meaning of lane, contract, baton, lease, resource, provider, harness, and scheduler before operating a multi-resource setup.

Start

```
npm run status
npm run up
npm start
```

Run `status` first for read-only inspection. `up` is health-first and starts only explicit configured recipes. The TUI then shows lane, work, provider, resource, PTY and baton state. With no config, infrastructure displays `UNCONFIGURED`; this is safe and expected.

Deployment patterns

Agent Control is normally an operator-facing process, not a public web service. Use one of these patterns:

- local workstation: TUI, repositories and optional local providers on one machine;
- controller plus workers: TUI and durable state on a controller, configured SSH/Orca execution on workers;
- API-only: no managed local models or workers, only explicitly configured remote provider APIs;
- Android-assisted: any of the above plus an optional configured Termux resource.

Keep `.agent-control/` on durable local storage with permissions restricted to the operator. Run the interactive TUI inside a persistent terminal multiplexer when disconnect/reconnect is required. Do not expose node health/job endpoints to an untrusted network; use loopback plus an authenticated transport.

For boot-time configured-service recovery, a service manager may invoke `npm run up` from the installation directory. Use a dedicated operating-system account and an explicit config path. Treat the example as a template, not an instruction to deploy automatically:

```
[Service]
Type=oneshot
WorkingDirectory=/srv/agent-control
Environment=AGENT_CONTROL_CONFIG=/etc/agent-control/config.json
ExecStart=/usr/bin/npm run up
ExecStop=/usr/bin/npm run down
```

```
RemainAfterExit=yes
```

The TUI remains an interactive operator session. A service manager must not be given broader privilege merely to launch configured services.

Configuration example

```
{
  "schemaVersion": 1,
  "resources": [
    {"id": "controller-a", "platform": "linux", "transport": {"type": "local"}, "capabilities": ["control-plane"]},
    {"id": "worker-a", "platform": "linux", "transport": {"type": "ssh", "host": "worker.example", "port": 2207, "user": "operator"}, "capabilities": ["harness.codex"]}
  ],
  "providers": [
    {"id": "api-a", "kind": "responses", "baseUrl": "https://api.example.test/v1", "qualificationModel": "qualified-model", "costClass": "metered"}
  ],
  "services": [],
  "lanes": [{"id": 1, "name": "Primary", "cwd": ".", "priority": 1, "mode": "auto"}]
}
```

URLs must not contain usernames/passwords. Do not add token, password, secret or API-key fields; validation rejects them. A hostname is only transport metadata and cannot prove a lease, owner or recovered execution.

TUI controls

Key	Action
Tab	select the next lane
I / Enter	open command input
T	inspect discovered PTYs
G	probe configured providers
Y	run the configured Responses proof
W	inspect Work Queue detail
D	inject the isolated demo workload
X	probe the configured Android resource
Z	request allow-listed Android recovery
A	toggle Android recovery AUTO/MANUAL
R	request capability/provider substitution
P	pause/resume and checkpoint
Q / Ctrl-C	persist and quit

Monitor

```
npm run status
npm run qualify
```

Monitor lane status, baton revision/health, ready/review queue counts, resource/provider health, assigned versus total PTYs, execution recovery state and context/provenance references. Qualification creates timestamped JSON beneath [qualification-results/](#); archive selected evidence only after reviewing it for secrets/private topology.

Healthy transport is not proof of original execution identity. After disconnect/restart, allow autonomous resume only after the recovery handshake matches task, execution/session, resource, repository, worktree, branch, nonce, lease generation, ownership generation and command identity as applicable.

Normal task workflow

1. Create or select a lane and set its objective, constraints, repository and acceptance tests.
2. Let the scheduler resolve capabilities, provider and resource, or lock those choices in MANUAL mode.
3. Verify the lease and ownership generation before execution/input.
4. Checkpoint at meaningful tool/state boundaries; keep the baton compact and current.
5. Inspect Git diff, tests, artifacts and optional context sources before handoff or completion.
6. For competing attempts, keep investigators independent, send their evidence to a judge, record dissent and let Agent Control select the winner.
7. Cancel/clean up losing executions through the provider contract; do not let the substrate choose continuation.

Execution and recovery states

- starting/running: execution identity has been established and is currently proven;
- paused/human-owned: work is deliberately fenced from normal agent input;
- completed/failed/cancelled: terminal outcomes with retained evidence;
- disconnected/recovering: continuity is being established, not assumed;
- unknown: original identity cannot be proved, so autonomous input and continuation are blocked.

Human takeover

Takeover always wins. It increments the Agent Control ownership generation, rejects agent input and leaves the terminal human-owned after adapter reconnect/restart. Return ownership deliberately; a stale writer never regains permission automatically.

Orca-backed execution

Configure Orca as the selected execution provider only after local qualification. Agent Control calls the narrow interface for start, status, reconnect, input, pause, resume, cancel, output, diff and cleanup. Keep the built-in executor as fallback through 3.0 qualification. Do not expose Orca-native scheduling or ownership controls directly to lanes.

This repository does not ship the Orca runtime, a service definition, or an Orca credential. See [Integrations and credentials](integrations-and-credentials.md) for the boundary.

Shared context and consensus

Attach existing read-only shared URLs as context sources. Never create or broaden a public share without explicit approval. Routing starts with a baton, adds diff/tests when needed, loads selected shared-thread sections only when valuable, and escalates disputes to independent agents plus a judge. Missing/expired context never blocks Git/baton recovery.

Batons are compact persisted lane records, not automatically generated cross-model summaries. The library has handoff/clone operations, but the TUI does not currently expose them, and no code negotiates a destination model's context window or rewrites Codex sub-agents. The reusable scheduler can allocate explicit work items by capability and load, but it is not yet wired as a general multi-LLM prompt router.

Android

Add a configured Android resource, supply its existing credential in the named environment variable, and run:

```
npm run provision:android
```

The mission pauses for explicit administrator approval if ADB installation is required, explicit human wireless-pairing approval, and explicit reboot-test approval. The node is loopback-bound by default and accepts only the read-only Android log job. Provisioning does not install an LLM or log the operator into Codex/OpenAI; it installs and verifies Android transport and boot prerequisites. See [Integrations and credentials](integrations-and-credentials.md) for the ordered operations and token boundary.

Stop and recover

```
npm run down
```

`down` stops only processes recorded as Agent-Control-owned. It does not stop unrelated listeners or services. Persisted state remains in `.agent-control/`. On restart, Agent Control reconciles objective state and may classify an unprovable execution as `UNKNOWN`; resolve that state manually or through a validated reconnect.

Upgrade and rollback

Before upgrading, checkpoint/pause work, preserve the state directory, verify the target tag/SHA and run the full check. Version 3.0.1 does not delete the fallback executor. Roll back source to the previously verified tag and restore only compatible persisted state; never claim execution survival without identity validation.

Backup, evidence and security

Back up the persisted state directory only after reaching safe checkpoints. Keep runtime credentials outside that backup unless the approved secret manager is included separately. Preserve evidence hashes, commit SHAs and test output; redact credentials and private topology before publication.

Provider text, shared threads and agent conclusions are untrusted inputs. Weight executable/test evidence first, repository evidence second, authoritative external sources third, and interpretations/assertions last. Three agents repeating one unsupported assumption do not make it true.

Review configured start commands as privileged policy. Agent Control never scans for similarly named services, guesses ports, kills unknown listeners, creates credentials, publishes conversations or broadens network/firewall access.

Troubleshooting

- **UNCONFIGURED**: create or point to a valid config file.
- provider unavailable: check its configured health URL, then run the functional proof; advertised health alone is insufficient.
- Android disconnected: validate configured transport separately from node health.
- execution **UNKNOWN**: do not resume automatically; compare persisted identity evidence.
- context unavailable: continue from baton, Git and tests; record source accessibility failure.
- **down** leaves a process: it was not recorded as Agent-Control-owned; inspect it manually rather than killing it through Agent Control.

Release validation

```
npm install
npm run typecheck
npm run check:bootstrap
npm run check:neutrality
npm test
npm run check
npm run qualify
git diff --check
```

Review the infrastructure audit, final string scan, changed-file list and secrets check before committing/tagging.
A source release is not a deployment.